Hackathon Challenge: "SwiftPay" Real-Time Payment Ledger

1. Project Overview

Scenario: You are building the core of a new fintech platform, **SwiftPay**. The goal is to create a resilient, scalable system that handles peer-to-peer (P2P) money transfers. The system must ensure data consistency, handle high-volume transactions via caching, and provide real-time audit logs for financial reporting.

2. Technical Stack (Mandatory)

- **Language:** Java 21/25 (Spring Boot or Quarkus)
- **Database:** PostgreSQL (Transactions)
- **Messaging:** Apache Kafka (Event-driven updates)
- **Caching:** Redis (Idempotency and balance lookups)
- **Documentation:** Swagger/OpenAPI
- **Infrastructure:** Docker & Kubernetes (Local or Minikube)
- **CI/CD:** GitHub Actions (Workflow for build and test)

3. Functional Requirements

Service A: Transaction Gateway (REST API)

- **POST /v1/payments:** Accepts a payment request (sender_id, receiver_id, amount, currency).
- **Idempotency:** Use **Redis** to ensure the same transaction_id isn't processed twice within a 24-hour window.
- **Validation:** Ensure the sender has enough balance (checked against the Ledger service or a cached value).
- **Workflow:** Save the initial request to PostgreSQL with a PENDING status and emit a PaymentInitiated event to **Kafka**.

Service B: Ledger Service (Consumer & Processor)

- **Kafka Listener:** Consume Payment Initiated events.
- **Atomic Operations:** Perform the balance transfer (Debit/Credit) within a database transaction.
- **Status Update:** Emit a PaymentCompleted or PaymentFailed event back to Kafka.
- **Reporting:** Expose a GET endpoint to fetch the transaction history for a specific user.

Service C (Bonus): Analytics Worker

- **OLAP Integration:** Consume PaymentCompleted events and write them to **ClickHouse** (or a mock analytics table) for real-time volume monitoring.

4. Non-Functional Requirements

1. **API Standards:** All endpoints must be documented using Swagger/OpenAPI. Use proper HTTP status codes and standard error response objects.
2. **Resilience:** Implement a retry mechanism for Kafka consumers (if the DB is temporarily down).
3. **Observability:** Implement a Health Check endpoint (/health) and basic logging.
4. **Containerization:** Provide a Dockerfile for each service and a docker-compose.yml (or K8s manifests) to spin up the entire ecosystem (App + Postgres + Kafka + Redis).
5. **CI/CD:** Create a GitHub Actions workflow that:
    - Compiles the Java code.
    - Runs Unit & Integration tests.
    - Builds the Docker image.

5. Hackathon Timeline (2-Day Effort)

Foundation

- Project scaffolding, API design with OpenAPI, and DB schema setup.
- Implementation of **Service A** (REST endpoints + Postgres integration).
- Setup Kafka brokers and implement the Producer logic in Service A and Consumer logic in **Service B**.

Polish, DevOps, and Performance

- Implement Redis-based idempotency and caching.
- Writing Unit and Integration tests (using Testcontainers if possible).
- Dockerization and creating the GitHub Actions pipeline.
- Performance tuning (Identify a bottleneck using a simple load test tool like Jmeter or K6).
- Documentation and final README submission.
- Perform a load test at **250 TPS** for a total of **1 million transactions**, and provide the resulting **PCAP trace**.

6. Submission Criteria

- **Code Quality:** Clean architecture (separation of layers), modular design, and meaningful variable naming.

- **Functionality:** Does the end-to-end payment flow work? Does it handle "insufficient funds"?
- **DevOps Readiness:** Does docker-compose up start the whole environment successfully?
- **Error Handling:** How does the system handle a Kafka outage or a Database constraint violation?
- **Github Repo for Review**